

Inter Process Communication

Processes in an operating system often need to work together. They often need to read and write the same data. Create other processes or each other using `fork()`, and be aware of each other e.g. when is certain process going to finish.

Because of this, processes need a way to communicate with each other. This is called Inter-Process Communication (IPC).

IPC tries to answer two big questions:

How can processes send or exchange data?

Example: One process generates numbers, another process calculates their sum → they need to share data.

How can processes safely access shared resources?

Example: Two processes want to write to the same file → they must not overwrite each other

Critical Section:

When processes run at the same time (parallel or almost parallel), they sometimes need to use shared data — like the same file, the same variable, the same memory area, etc.

Inside every process, there are two types of code sections:

Critical Code:

Code of the process that reads and writes into data, change something inside the shared memory or modify something in the file.

Only one of the processes can run the code that modifies these shared files, if it happens that 2 processes or more execute code that manipulates shared data. Data corruption happens.

Un-Critical Code:

Code inside the process that only reads files, data, or only uses data that are specific to this process.

Mutual Exclusion:

The OS makes sure that only one process at a time can execute critical code.

Critical Section example Spooler directory:

A process X runs a code to save a file in the directory, the process starts then before it saves the file a process Y that has the same critical code fully runs and saves a file to a directory. The process X then proceeds and saves the same file in the same directory making the files saved by Y replaced by the files done by the process X. This is called

Race Condition:

The main idea and the problem happens when one a process interrupts and use the data before the other process finish writing (updating the data).

2 Processes that tries to access and modify the shared data at the same time.

The processes acts correctly the problem is with interference. The result of the final 2 processes depends on each case.

Communication Vs Cooperation:**Communication:**

There is no shared data, the IPC here happens when process X sends a copy of its own data to process Y. Process Y then can use this data and modify it, if we changed this data inside process X the copy we passed to process Y remains unaffected.

There is no shared memory accessed.

Cooperation:

2 or more processes are working using shared data, files or memory.

Both can read and write on this data, access must be synchronized to prevent race conditions.

Synchronisation:

Its when processes work together in a way that ensures that tasks happen in proper way, shared data is not messed up by multiple processes accessing it at the same time.

Wait(): sometimes processes need to wait for each other to finish for instance process B needs to access shared data but only and only after process A has finished it so process B waits for Process A to finish using the wait() in linux.

Wait for results: sometimes the process need to wait for other process to give a certain result so that it can continue.

Execution Order: it makes sure that the processes are being ran in their order

Critical section and race condition: makes sure that only one process can access their critical section at a time to prevent race conditions.

Synchronisation by Signaling – Busy Waiting:

There are 2 processes lets assume X and Y.

Y need a signal S that might be a variable, a file change a directory or anything. So while Y waits for the signal S from X, it keeps checking S from X which uses and consumes CPU while Y is not doing anything, once Y receives S it continues. This is also called polling.

Synchronisation by Signaling – Signal and Wait:

Lets assume the same (X, Y, S) , process Y waits and gives away the CPU (blocked, waiting) until X finishes a certain task and send a S to Y as a wake up signal. Y then is back to being executed.

Pro: CPU is not wasted and this is very time efficient.

Con: only one process can wait for one signal.

lock and unlock:

The main idea is to make sure only and only one process can execute its critical code, for example; Process A and Process B work on some shared data. Process A wants to manipulate this data, Process A asks for lock() then acquires it and start executing code on this data. If Process B asks for the lock() or try to execute critical code it will be block. Process B can only get the lock when process A call to unlock().

Unix-like OS system calls mentioned

These are ways processes can synchronize or wait:

sigsuspend() → Waits for a signal while temporarily replacing signal masks,
sigsuspend(*mask) wait for a signal that is not inside the mask to unlock the process

kill() → Sends a signal to a process

pause() → Waits for any signal

sleep() → Waits for a fixed amount of time

No execution order is guaranteed locking and unlocking does not interfere with scheduling

Locking ensures that critical sections don't overlap, but it doesn't decide which process goes first.

That depends on scheduling and which process acquires the lock first.

Beware of locking indefinitely which might cause deadlocks.\

Problems Induced by Blocking:

Starvation:

If a process hold the lock it stops all other processes that needs this data and depend on it this might starve these processes since they all depend on Process A to release the key.

The same issue happens with priority scheduling,

Deadlocks:

A deadlock occurs when two or more processes are each waiting for a resource that the other process holds, so none of them can proceed.

Imagine two processes, P1 and P2, and two resources, R1 and R2:

1. P1 acquires R1.
2. P2 acquires R2.
3. P1 now wants R2, but it's held by P2 → P1 blocks.
4. P2 now wants R1, but it's held by P1 → P2 blocks.

We can avoid this by:

Resource Graphs • Petri Nets • Deadlock detection by matrice

A deadlock occurs when all these four conditions happen at the same time:

1. Mutual Exclusion
 - Some resources can only be used by one process at a time and cannot be shared.
2. Hold and Wait
 - A process already has one resource and is asking for another that is held by someone else.
3. No Preemption
 - The operating system cannot take a resource away from a process.
 - The process must release it voluntarily.
4. Circular Wait
 - There is a loop of processes where:
 - P1 waits for a resource held by P2
 - P2 waits for a resource held by P3 ...
 - The last process waits for a resource held by P1